compiled JavaScript or set an *X-SourceMap* header if the file is served over HTTP. You add a comment to the file like this:

```
echo '\n//@ sourceMappingURL=sourcemaps.js.map' >> sourcemaps.js
```

Now, consider that `'Using source maps'` error that your program throws—how does it appear? When you run the program in an environment that doesn't understand source maps, you'll see the error reported in the compiled JavaScript file:

```
Uncaught Error: Using source maps
double -- sourcemap.js:5
```

That's not very useful. An exception in compiled JavaScript when you're writing your program in CoffeeScript is difficult to comprehend. In contrast, with source maps you'll see the error reported in your original CoffeeScript program:

```
Uncaught Error: Using source maps
double -- sourcemaps.coffee:2
```

It gets better. Not only will you see errors on appropriate lines, but you'll also be able to harness the full power of your favorite IDE on your CoffeeScript program. Stepping through and setting breakpoints will be done in your CoffeeScript source, and not in the generated JavaScript.

Enabling source maps in your browser

As with most configuration options, each browser will have a slightly different way of enabling source maps. As an example, Chrome 27 allows you to enable source maps via a check box in the settings for the developer tools. Consult the documentation for the relevant version of the browser (or other runtime) you're using for details on how to enable source maps.

It's still early days for source maps, so you'll need to consult the documentation for your individual runtime to get them working. If you like